

Recurso de profundización – Semana 3

Título del recurso: Construcción de un *portfolio responsivo* con CSS Grid y Flexbox

Descripción del recurso: Este recurso es una guía práctica extendida que muestra cómo aplicar Flexbox y Grid junto con Media Queries para crear un layout de *portfolio responsivo*, integrando lo visto en los subtemas 3.1 a 3.4.

Objetivo: Profundizar en la comprensión del diseño adaptable mediante un caso práctico completo.

Paso 1. Estructura básica en HTML

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>Portfolio Responsivo</title>
  <link rel="stylesheet" href="styles.css" />
</head>
<body>
  <header class="header">
    <h1>Mi Portfolio</h1>
    <nav class="nav">
      <a href="#">Inicio</a>
      <a href="#">Proyectos</a>
      <a href="#">Contacto</a>
    </nav>
  </header>

  <main class="container">
    <section class="about">
      <h2>Sobre mí</h2>
      <p>Soy estudiante de desarrollo web, apasionado por el diseño adaptable y accesible.</p>
    </section>

    <section class="projects">
      <article class="project">Proyecto 1</article>
      <article class="project">Proyecto 2</article>
      <article class="project">Proyecto 3</article>
      <article class="project">Proyecto 4</article>
    </section>
  </main>

  <footer class="footer">
    <p>© 2025 - Mi Nombre</p>
  </footer>
</body>
</html>
```

Paso 2. Estilos base con CSS (Mobile First)

```
* { box-sizing: border-box; margin: 0; padding: 0; }

body {
  font-family: Arial, sans-serif;
  line-height: 1.5;
  color: #222;
  background: #f9f9f9;
}
```

```

/* Header con Flexbox */
.header {
  display: flex;
  flex-direction: column;
  align-items: center;
  background: #0366d6;
  color: #fff;
  padding: 1rem;
}
.nav a {
  color: #fff;
  text-decoration: none;
  margin: 0 .5rem;
}

/* Contenido principal */
.container {
  padding: 1rem;
}

/* Grid de proyectos (1 columna por defecto) */
.projects {
  display: grid;
  grid-template-columns: 1fr;
  gap: 1rem;
  margin-top: 1rem;
}
.project {
  background: #fff;
  border: 1px solid #ddd;
  border-radius: .5rem;
  padding: 1rem;
  text-align: center;
}

```

Paso 3. Adaptación con Media Queries

```

/* Tablet */
@media (min-width: 768px) {
  .header { flex-direction: row; justify-content: space-between; }
  .projects { grid-template-columns: repeat(2, 1fr); }
}

/* Escritorio */
@media (min-width: 1200px) {
  .projects { grid-template-columns: repeat(4, 1fr); }
  .container { max-width: 1200px; margin: auto; }
}

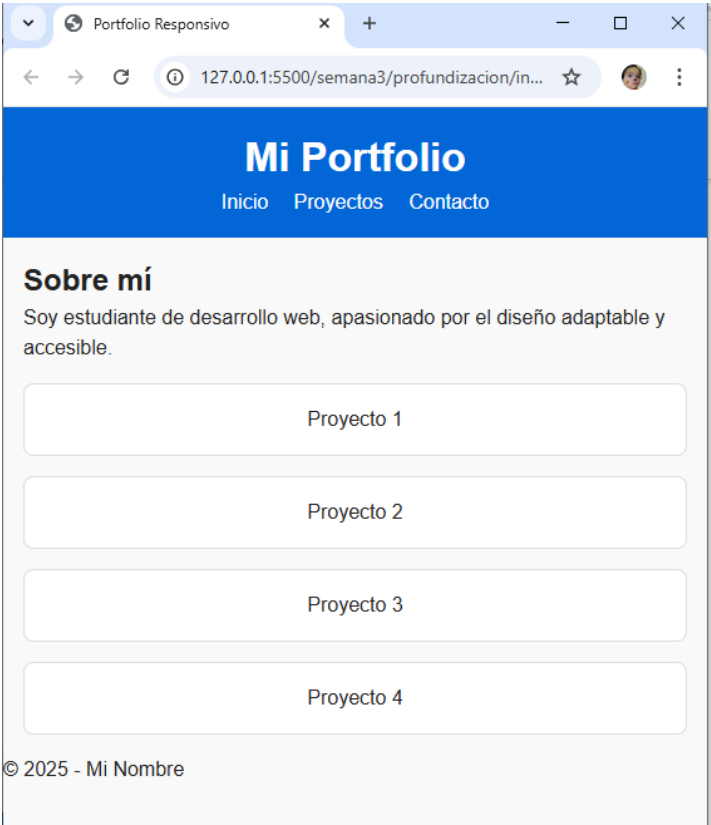
```

Paso 4. Reflexión guiada

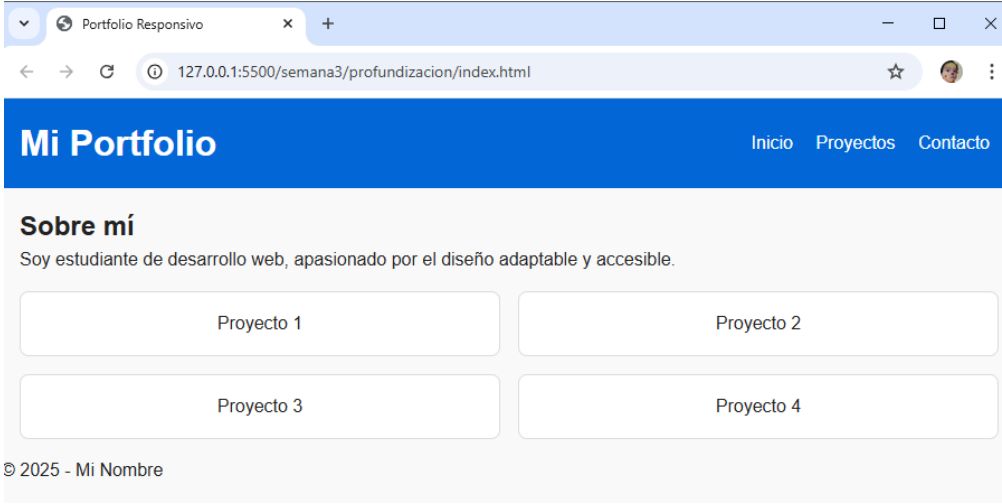
- ¿Qué papel juega Flexbox en el header?
Permite alinear logo y navegación horizontalmente en pantallas grandes.
- ¿Por qué Grid es mejor para los proyectos?
Porque distribuye automáticamente en columnas, ajustando el número según el ancho.
- ¿Qué hubiera pasado si usáramos solo float o medidas fijas en píxeles?
El diseño se rompería en móviles o pantallas grandes.

Ejemplo visual del resultado

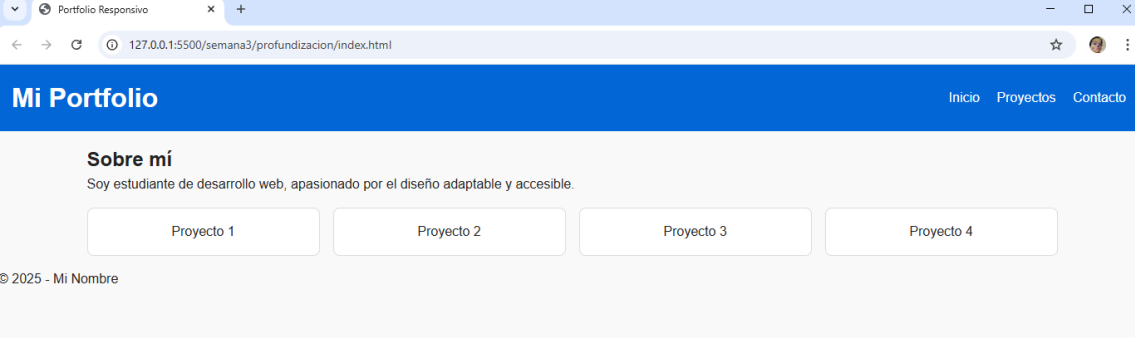
Móvil (1 col):



Tablet (2 col):



Escritorio (4 col):



Enlace externo recomendado

Título del enlace relacionado: CSS Grid — Guía completa (MDN Web Docs)

Descripción del enlace relacionado: Recurso oficial de MDN que profundiza en el uso de Grid Layout con ejemplos prácticos y casos de uso avanzados.

Enlace: https://developer.mozilla.org/es/docs/Web/CSS/CSS_grid_layout